MATH10017 Assessed Coursework 1 (2026/27)

Matteo Fasiolo

Use of AI: **Prohibited**

Context

A **grayscale image** is simply a rectangular grid of numbers. Each number is the intensity, or brightness, of one **pixel** (picture element). Here we use integers between 0 and 255, where 0 means black and 255 means white. An image with n_{row} rows and n_{col} columns is therefore an $(n_{\text{row}} \times n_{\text{col}})$ matrix of integers.

Most of the operations that photo editing software performs on an image are, underneath, very simple loops over this grid. Making a picture darker or brighter means changing every entry by the same rule. Mirroring a picture means moving entries around. Smoothing a picture, to remove graininess, means replacing each entry by an average of the entries around it.

In this coursework you will write a small C program that does exactly these things. The image is provided to you, so you will not need to read any file from disk. You will need only the C features covered in the lectures up to Week 5: arrays, loops, functions, pointers and dynamic memory allocation.

1 The files provided

You are given two files, which must be kept **in the same folder**:

- `image_data.h`, which contains the image. **Do not modify this file.**
- `xy12345.c`, a template that you should rename using your own UoB username (e.g. `ab12345.c`) and complete.

The template already starts with the lines

```
#include <stdio.h>
#include <stdlib.h>

#include "image_data.h"
```

You have already met `#include <stdio.h>`, which makes `printf` available, and `#include <stdlib.h>`, which makes `malloc` and `free` available. The third line is new, but does the same job: writing the file name in **double quotes**, rather than in angle brackets, simply tells the compiler to look for the file in the current folder rather than among the standard system libraries. The effect is as if the contents of `image_data.h` had been pasted at that point of your program.

To compile and run your program, open a terminal in that folder and type

```
gcc -Wall xy12345.c -o cw1
./cw1
```

(use `.\cw1` instead of `./cw1` on Windows). Note that we give the executable no extension: on Windows `gcc` adds `.exe` by itself, so the same command works on Windows, Linux and Mac OS. The `-Wall` flag asks the compiler to report all warnings, which is a good habit: warnings often point at real bugs, such as a function that can reach its end without returning a value. As in the lectures, we declare `main` as `int main(void)` and end it with `return 0;`. A good solution compiles without warnings.

2 How the image is stored

The file `image_data.h` defines three things:

```
#define NROW 32
#define NCOL 32

int IMAGE[NROW*NCOL] = { ... };
```

So `NROW` and `NCOL` are the number of rows and columns of the image, and can be used anywhere in your program as if they were the numbers 32 and 32. The image itself is the array `IMAGE`, which holds $32 \times 32 = 1024$ integers between 0 and 255.

As you saw in the lectures, a matrix has to be flattened into a one-dimensional array. Here the image is stored in **row-major** order: the first `NCOL` entries of `IMAGE` are the first row of the image, the next `NCOL` entries are the second row, and so on. We use **zero-based indexing** throughout, so the rows are numbered $0, 1, \ldots, n_{\text{row}} - 1$ and the columns $0, 1, \ldots, n_{\text{col}} - 1$.

Important. All the functions below take the number of rows and columns as arguments, and they must work for **any** image size, not only for 32×32. Do not use `NROW` and `NCOL` inside your functions: use the arguments instead. See the note on hard-coding at the end of this document.

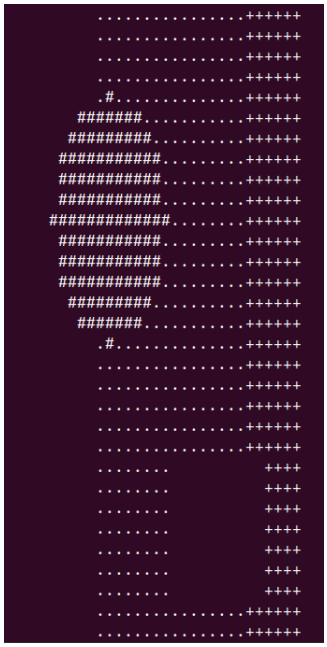


Fig. 2.1: The original matrix, defined in image_data.h, rendered via ASCII art.

Your tasks

Part I: displaying an image

1. Write a function

   ```
   int idx(int i, int j, int ncol)
   ```

   that returns the position, within the flattened array, of the pixel on row `i` and column `j` of an image with `ncol` columns stored in row-major order. For example, `idx(0, 0, 32)` must return 0 and `idx(2, 5, 32)` must return 69.

   From now on, whenever you need the pixel on row `i` and column `j` of an image `img`, you should write `img[idx(i, j, ncol)]`.

2. We will display an image in the terminal as "ASCII art", by printing one character per pixel, with darker pixels drawn using lighter characters. Write a function

   ```
   char pixel_char(int v)
   ```

   that takes an intensity `v` and returns the character to be printed, according to the following table:

intensity `v`	character
$0 \leq v \leq 50$	`' '` (a space)
$51 \leq v \leq 101$	`'.'`
$102 \leq v \leq 152$	`'+'`
$153 \leq v \leq 203$	`'*'`
$204 \leq v \leq 255$	`'#'`

   Recall that a single character is written between single quotes in C, for instance `'#'`, and that it can be printed with the `%c` specifier, as in `printf("%c", 'A')`.

3. Write a function

   ```
   void print_image(int img[], int nrow, int ncol)
   ```

   that prints the whole image, one row of characters per row of the image, using `pixel_char`. Use it to print the image `IMAGE`. You should obtain exactly the picture in Figure 2.1 (a bright disc on the left, and a dark rectangle at the bottom right, on a background that gets brighter from left to right).

Part II: simple pixel operations

4. Write a function

   ```
   void minmax(int img[], int n, int *mn, int *mx)
   ```

   where `n` is the total number of pixels of the image (that is, `nrow*ncol`). The function does not return anything. Instead, it must write the **smallest** intensity of the image into the variable pointed to by `mn`, and the **largest** intensity into the variable pointed to by `mx`.

   You should call it like this:

   ```
   int mn, mx;
   minmax(IMAGE, NROW*NCOL, &mn, &mx);
   printf("min = %d, max = %d\n", mn, mx);
   ```

5. Write the following two functions, which both modify the image **in place**, that is, they change the array they are given rather than creating a new one.

   (a) `void invert(int img[], int n)`

   which replaces every intensity `v` by `255 - v`. This turns a photographic negative into a positive, and vice versa.

   (b) `void flip_h(int img[], int nrow, int ncol)`

   which mirrors the image from left to right, so that column 0 is exchanged with column $n_{\text{col}} - 1$, column 1 with column $n_{\text{col}} - 2$, and so on.

   Print the image after inverting it, and print it again after flipping it, so that both effects can be seen. Take care that the two operations do not interfere with each other: the flipped picture you print should be a mirror image of the **original** picture, not of the inverted one. Explain in a comment how you dealt with this.

   Also report the minimum and maximum intensity of the inverted image, using your `minmax` function, and say in a comment why the two values you obtain are what you should have expected.

Part III: smoothing an image

Photographs are often smoothed, or "blurred", to remove graininess. The simplest way of doing this is the 3×3 **mean filter**: the new intensity of a pixel is the average of the intensity of that pixel and of its eight immediate neighbours, that is, the average of the nine pixels in the 3×3 square centred on it.

Pixels on the border of the image do not have eight neighbours. To keep things simple, a border pixel (one on row 0, on row $n_{\text{row}} - 1$, on column 0, or on column $n_{\text{col}} - 1$) is left unchanged.

Note that this operation **cannot** be done in place: if you overwrote each pixel as you went along, the averages computed later would use the new values rather than the original ones. You therefore need a second image to write the result into.

6. Write a function

   ```
   int *blur(int img[], int nrow, int ncol)
   ```

   which returns a **new** image, containing the result of applying the mean filter once to `img`. The input image must not be modified.

   Use integer division when averaging, so that the result stays an `int`: if `sum` is the sum of the nine neighbouring intensities, the new intensity is `sum / 9`.

   Apply your function to `IMAGE` and print the result.

7. Write a function

   ```
   int *blur_k(int img[], int nrow, int ncol, int k)
   ```

   which applies the mean filter `k` times in a row, each time to the result of the previous application, and returns the final image. As before, the input image must not be modified, and the image that is returned must be a freshly allocated one.

   Be careful with memory here: each call to `blur` allocates a new image. Explain in a comment how your function avoids leaking memory, and what your function returns when `k` is 0.

   Use `blur_k` to apply the filter 10 times to `IMAGE`, print the result, and report the minimum and maximum intensity of the smoothed image. In a comment, describe what has happened to the picture, and explain why the minimum and maximum have moved in the direction they have.

Marking criteria

Please submit a single C file named using your UoB username, for example `ab12345.c`. Do not submit `image_data.h`, and do not submit the compiled program.

Your program must compile with `gcc -Wall` and run to completion. A program that does not compile will lose a substantial number of marks, so if you cannot get one of the tasks to work, comment out the offending code and say so in a comment, rather than submitting a file that does not compile.

The marking scheme below is indicative and is intended only as a guide to the relative weighting of the different parts of the project:

1. Part I: 25 points
2. Part II: 30 points
3. Part III: 25 points
4. Correct use of memory and pointers: 10 points. Arrays are copied only when necessary, every dynamically allocated array is freed exactly once, allocated memory is not accessed after it has been freed, and so on.
5. Coding style: 10 points. Appropriate comments, indentation, variable names and code formatting, and a `main` function whose printed output is clearly laid out and easy to follow.

NOTE This project can be solved completely using only the C features and library functions we have used in the lectures and labs: arrays, loops, functions, pointers, and dynamic memory allocation with `malloc` and `free` (together with `printf` for output). A solution that relies on constructs or library functions that were not covered (for example additional standard-library headers, or ready-made functions for copying, sorting or searching an array) will lose marks, **even if it compiles and produces the correct output**.

NOTE Hard-coded solutions will be penalised. Imagine that you are asked to write a function that sums the elements of an array of length `n`. This is a hard-coded solution, because it only ever works for the one array we happen to be using today:

```
int sum_hard(int v[]){
    int out = 0;
    for(int i = 0; i < 1024; i++){
        out += v[i];
    }
    return out;
}
```

while this is an acceptable, general solution:

```
int sum_ok(int v[], int n){
    int out = 0;
    for(int i = 0; i < n; i++){
        out += v[i];
    }
    return out;
}
```

In particular, your functions must not use `IMAGE`, `NROW`, and `NCOL` internally, and must not assume that the image is square.

Plagiarism and AI use policy

While you can discuss the general strategy of your code with your classmates, you must code independently. Code sharing and/or co-developing will be treated as academic collusion. This is a serious offence, see here.

You should be able to write the code for this project only using the functions and libraries we have used so far in the course. The use of code found on the internet is discouraged. But, if you do use code found on the web or elsewhere, it should be limited to a very small section of your code and you will need to disclose its source in your comments. Otherwise, it will be regarded as plagiarism.

The use of AI tools is **prohibited** for this coursework.